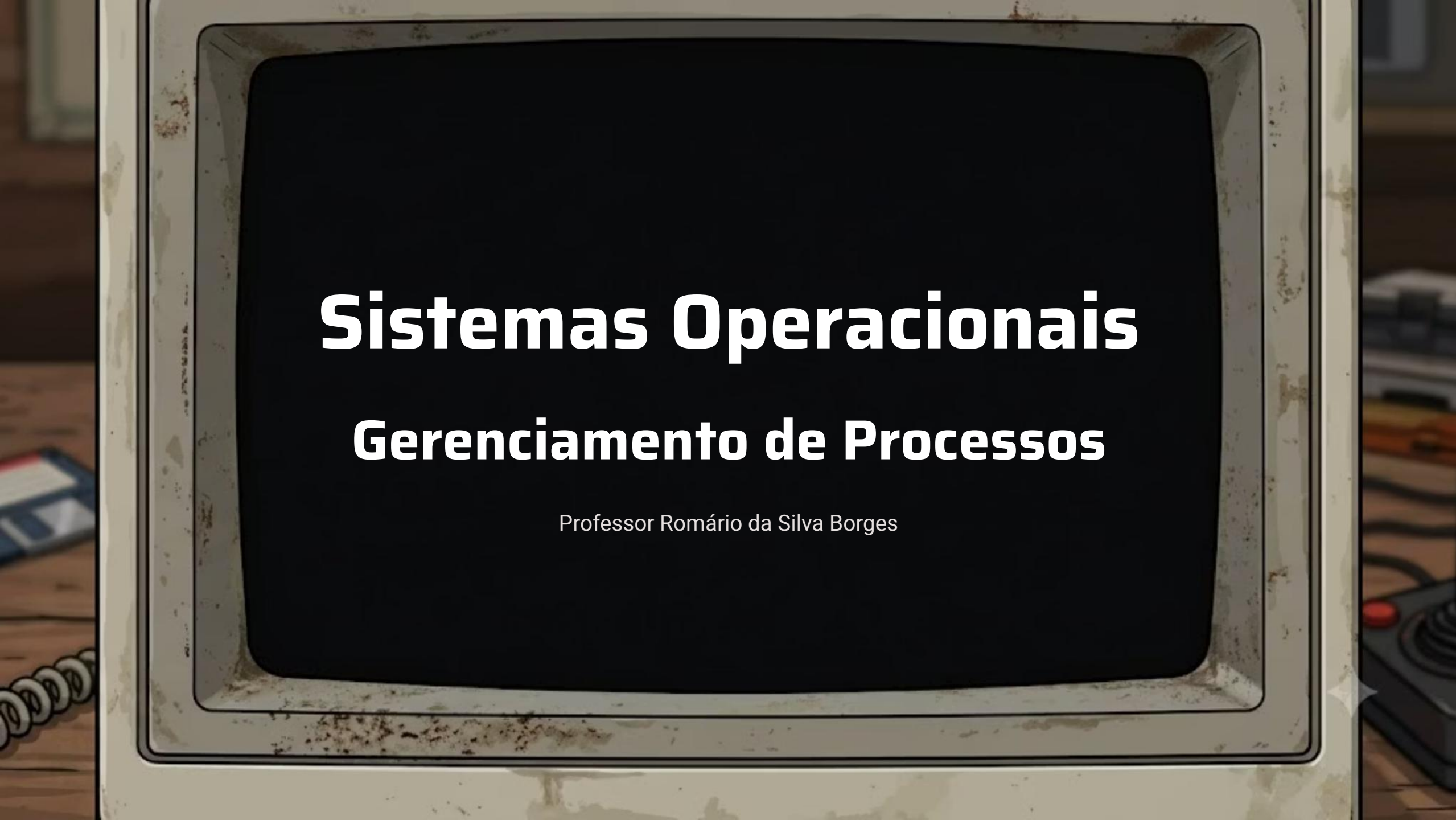




Sistemas Operacionais

Gerenciamento de Processos

Professor Romário da Silva Borges



? Se a CPU executa uma instrução por vez, como o sistema consegue rodar vários programas ao mesmo tempo?

Processos



BUSCAR



LAYOUTS



ELEMENTOS



TEXTO



FUNDO



UPLOADS

Pesquisar entre 1.000.000 imagens...

Layouts do Canva

NOVO



MIGO,
TÔ NA DÚVIDA.
MINI, MAXI
OU ARCOÍRES?

CACHORROSTYLE.COM



+ Adicionar nova página



- 75% +

Processo

Um processo é um **programa em execução**. Inclui código, dados, pilha e registradores e é a **unidade básica de execução** controlada pelo SO.

Programa × Processo

Programa

Entidade **passiva**, armazenada em disco

Processo

Entidade **ativa**, em execução na memória

 Um mesmo programa pode gerar **vários processos** simultaneamente



Componentes de um processo

Código

Instruções do programa

Dados

Variáveis globais e estáticas

Pilha

Chamadas de função e variáveis locais

Heap

Memória alocada dinamicamente

Registradores

Estado atual da CPU

Contador de programa

Próxima instrução a executar

Contexto do processo

O que é?

Todo o estado necessário para **continuar a execução** de um processo

O que inclui?

Registradores, **PC** (contador de programa), **SP** (ponteiro de pilha) e informações do SO

Para que serve?

Precisa ser **salvo e restaurado** a cada troca de contexto

Em sistemas operacionais, o **contexto de um processo é salvo no PCB** (*Process Control Block*, ou Bloco de Controle de Processo).

Tabela de processos / PCB

O que é o PCB?

Estrutura mantida pelo Sistema Operacional para cada processo ativo. Também chamado de **Bloco de Controle de Processo**.

O **PCB** guarda as informações necessárias para o sistema operacional pausar um processo e depois retomá-lo do mesmo ponto. Isso é essencial durante a troca de contexto (*context switch*).

Informações armazenadas

- Identificador do processo (PID)
- Estado do processo
- Contador de programa
- Registradores
- Prioridade
- Informações de memória
- Arquivos abertos
- Tempo de CPU

Gerenciamento de processo	Gerenciamento de memória	Gerenciamento de arquivo
Registros Contador de programa Palavra de estado do programa Ponteiro da pilha Estado do processo Prioridade Parâmetros de escalonamento ID do processo Processo pai Grupo de processo Sinais Momento em que um processo foi iniciado Tempo de CPU usado Tempo de CPU do processo filho Tempo do alarme seguinte	Ponteiro para informações sobre o segmento de texto Ponteiro para informações sobre o segmento de dados Ponteiro para informações sobre o segmento de pilha	Diretório-raiz Diretório de trabalho Descritores de arquivo ID do usuário ID do grupo

Troca de contexto



A troca de contexto permite que a CPU **alterne entre processos**, dando a impressão de execução simultânea.

Custo da troca de contexto



Trabalho inútil

Durante a troca, a CPU **não executa trabalho útil** do usuário



Eficiência

Trocas excessivas **reduzem o desempenho** do sistema



Quantum

O **quantum** (fatia de tempo) influencia diretamente esse custo

Criação de Processos

Quando processos são criados?



Inicialização do sistema

Processos essenciais iniciados com o SO (daemons, serviços)



Chamada de sistema

Um processo existente solicita a criação de outro via chamada ao SO



Solicitação do usuário

Usuário abre um aplicativo ou executa um comando



Tarefas em lote

Sistemas de lote disparam processos automaticamente

Criação no UNIX

`fork()`

Cria um **processo filho** — inicialmente uma cópia exata do pai

`exec()`

Substitui o programa em execução pelo novo programa desejado

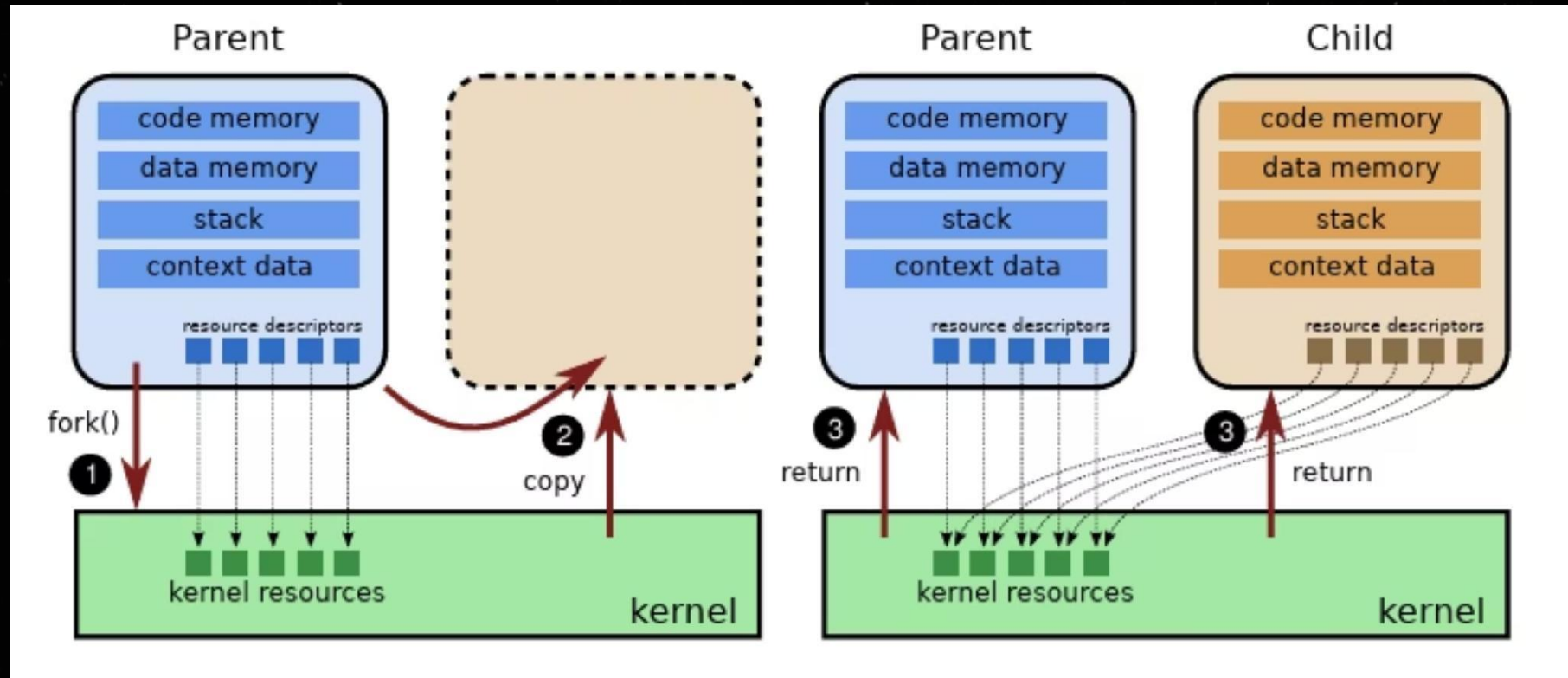
Redirecionamento

Entre o `fork` e o `exec` é possível redirecionar entrada e saída

Padrão POSIX — usado em Linux, macOS e outros sistemas Unix-like

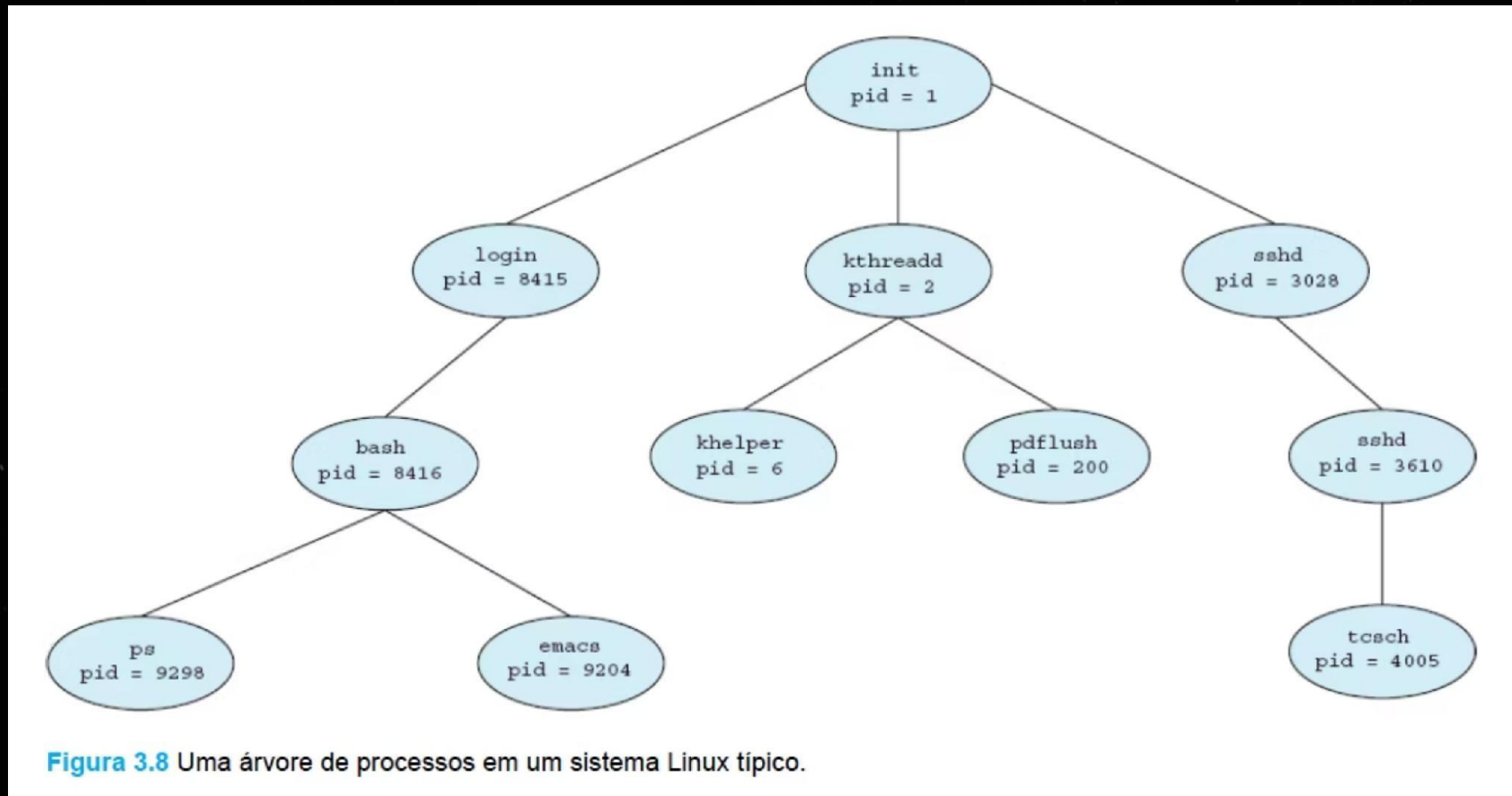
A memória do filho é inicialmente uma cópia da do pai (compartilhamento), até que o processo seja modificado.

Criação de um Processo em Unix



Ambos os processos acessam os mesmos recursos do núcleo, embora executem em áreas de memória distintas.

Criação de um Processo em Unix



Criação de um Processo em Unix

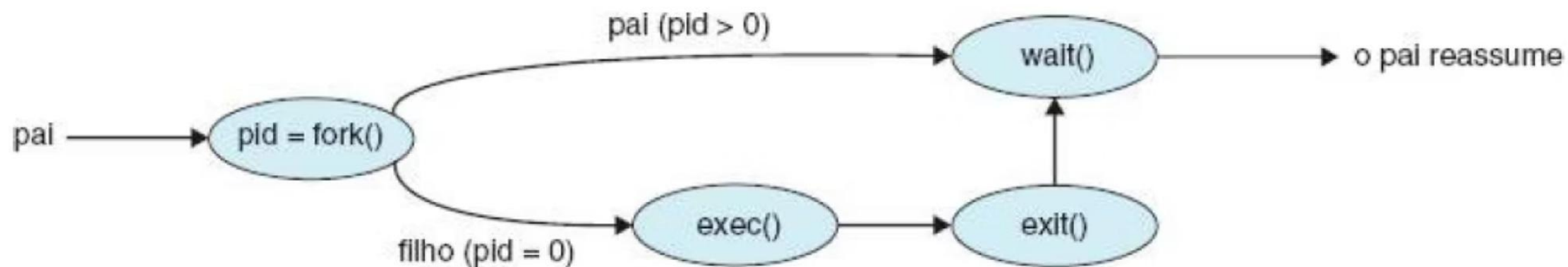


Figura 3.10 Criação de processo com o uso da chamada de sistema `fork()`.

Criação no Windows



`CreateProcess()`

Chamada de sistema que **cria um novo processo** do zero

Parâmetros

A chamada recebe vários parâmetros: programa, argumentos, atributos de segurança, etc.

Diferença para UNIX

O processo já é **criado com as características definidas** — não há separação fork/exec. **É criado e já executado.**

 Padrão Windows NT — utilizado em todo o ecossistema Windows moderno

Os espaços de endereço do pai e do filho são distintos desde o início, sem a necessidade de clonagem inicial.

Hierarquia de Processos



Não tem conceito de hierarquia. Quando um processo cria um filho, ele atribui um ID (Handle), mas o pai pode passar pra outro processo;



O init é raiz de tudo, e os filhos são criado como em árvores. Cada processo filho escolhe se aceita ou não os processos dos pais. Caso encerre o processo pai, ele é "adotado" por um acima (Init);

Término de Processos

Como um processo termina?



Saída normal

Processo conclui sua tarefa e encerra voluntariamente



Saída por erro

Processo encontra um erro tratável e encerra com código de erro



Erro fatal

Erro irrecuperável — ex.: divisão por zero, acesso inválido à memória



Cancelamento externo

Outro processo (com permissão) encerra o processo forçadamente

Chamadas de término



UNIX / Linux

- `exit()` – encerramento voluntário do processo
- `kill()` – envia sinal para encerrar outro processo



Windows

- `ExitProcess()` – encerramento voluntário
- `TerminateProcess()` – encerramento forçado externo



Após o término, o SO **libera todos os recursos** alocados pelo processo

Chamadas de término

Processo zumbi no Unix: Quando um Processo filho termina, ele permanece na tabela de Processos até o pai chamar o *Wait()*. Enquanto o pai não o chama, é conhecido como **Processo Zumbi**.

Caso o pai não chame o *Wait()* e finalize o processo, o que acontece com o filho zumbi?



Estados de Processos

Estados de um processo

 **Novo**

Processo sendo criado

 **Pronto**

Aguardando a CPU ser atribuída

 **Executando**

Usando a CPU neste momento

 **Bloqueado**

Aguardando evento externo (E/S)

 **Finalizado**

Encerrou a execução

Estado: Pronto

O que significa?

O processo **pode executar** — está apto, mas aguarda a CPU

Onde fica?

Inserido na **fila de prontos**, gerenciada pelo escalonador

Transição

Sai desse estado quando o escalonador faz o **despacho** → passa a Executando

Estado: Executando

O que significa?

O processo está **usando**
a **CPU** no momento atual

Restrição

Apenas **um processo** por
CPU pode estar nesse
estado

Saídas possíveis

Pode **terminar**, ser
preemptado (volta a Pronto)
ou **bloquear** (espera evento)

Estado: Bloqueado

O que significa?

O processo **aguarda um evento externo** que não depende da CPU

Exemplos de eventos

Leitura de disco, entrada de teclado, resposta de rede

Princípio

Processo **não deve ocupar a CPU** enquanto aguarda — libera para outros

Transições de estado



1. O processo é bloqueado aguardando uma entrada
2. O escalonador seleciona outro processo
3. O escalonador seleciona esse processo
4. A entrada torna-se disponível



As transições definem **como o SO movimenta os processos** entre os estados ao longo do tempo.

Multiprogramação e Pseudoparalelismo

Monoprogramação × Multiprogramação

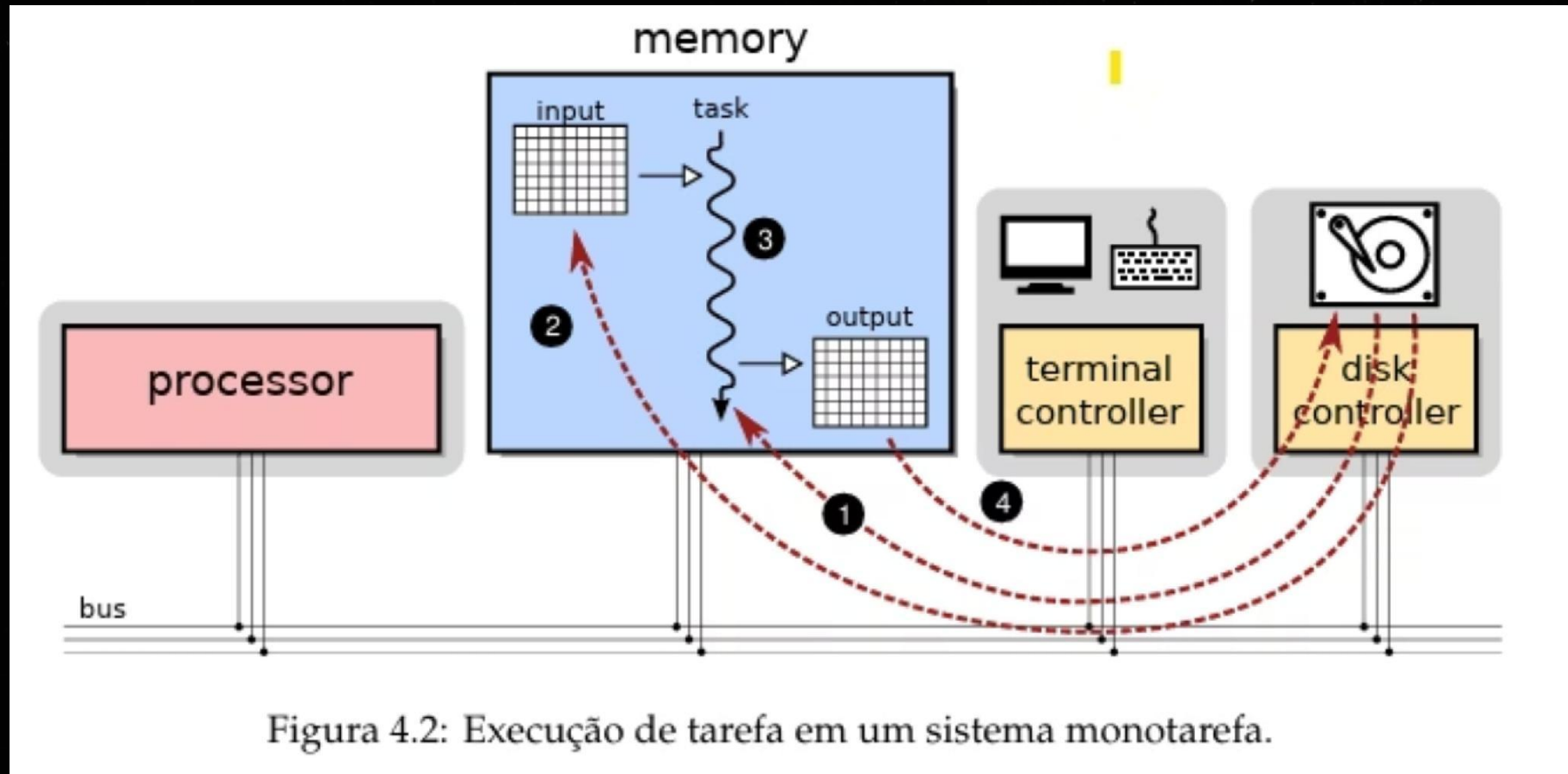
Monoprogramação

Um programa executa até terminar. CPU fica ociosa durante E/S.

Multiprogramação

Processos **alternam** o uso da CPU. Melhor aproveitamento dos recursos.

Monoprogramação



Multiprogramação e o Pseudoparalelismo

Como funciona?

A CPU **alterna entre programas** em pequenos intervalos de tempo

Percepção do usuário

O usuário percebe uma **falsa simultaneidade** — parece que tudo roda ao mesmo tempo

Benefício

Enquanto um processo espera E/S, outro **aproveita a CPU**

Uma CPU por vez

Apenas **um processo executa de cada vez** em uma única CPU

Ilusão

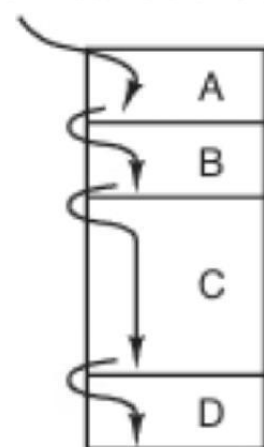
A **troca rápida** cria a impressão de execução paralela

≠ Paralelismo real

Diferente do paralelismo em **múltiplas CPUs** — aqui há execução simultânea de fato

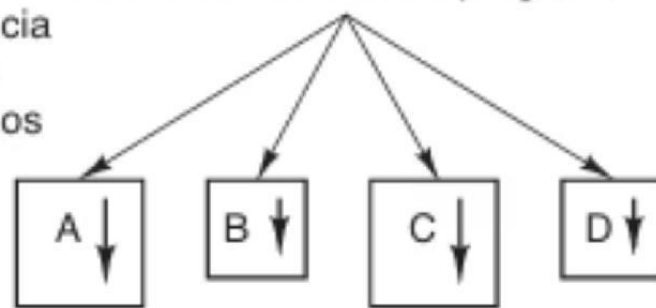
(a) Multiprogramação de quatro programas. (b) Modelo conceitual de quatro processos sequenciais independentes. (c) Apenas um programa está ativo de cada vez.

Um contador de programa



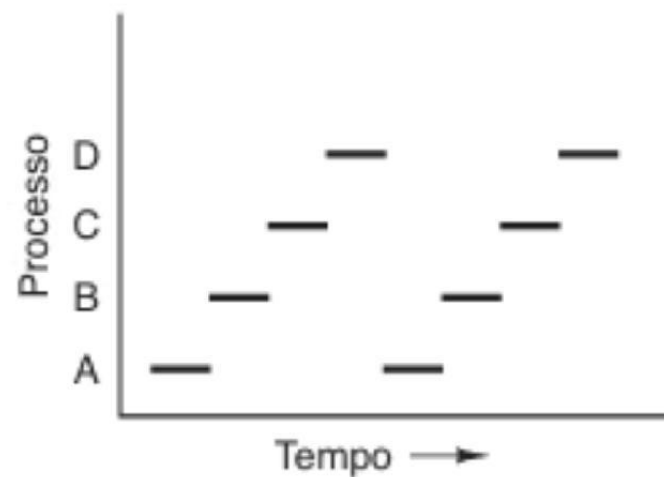
Alternância
entre
processos

Quatro contadores de programa



(a)

(b)



(c)

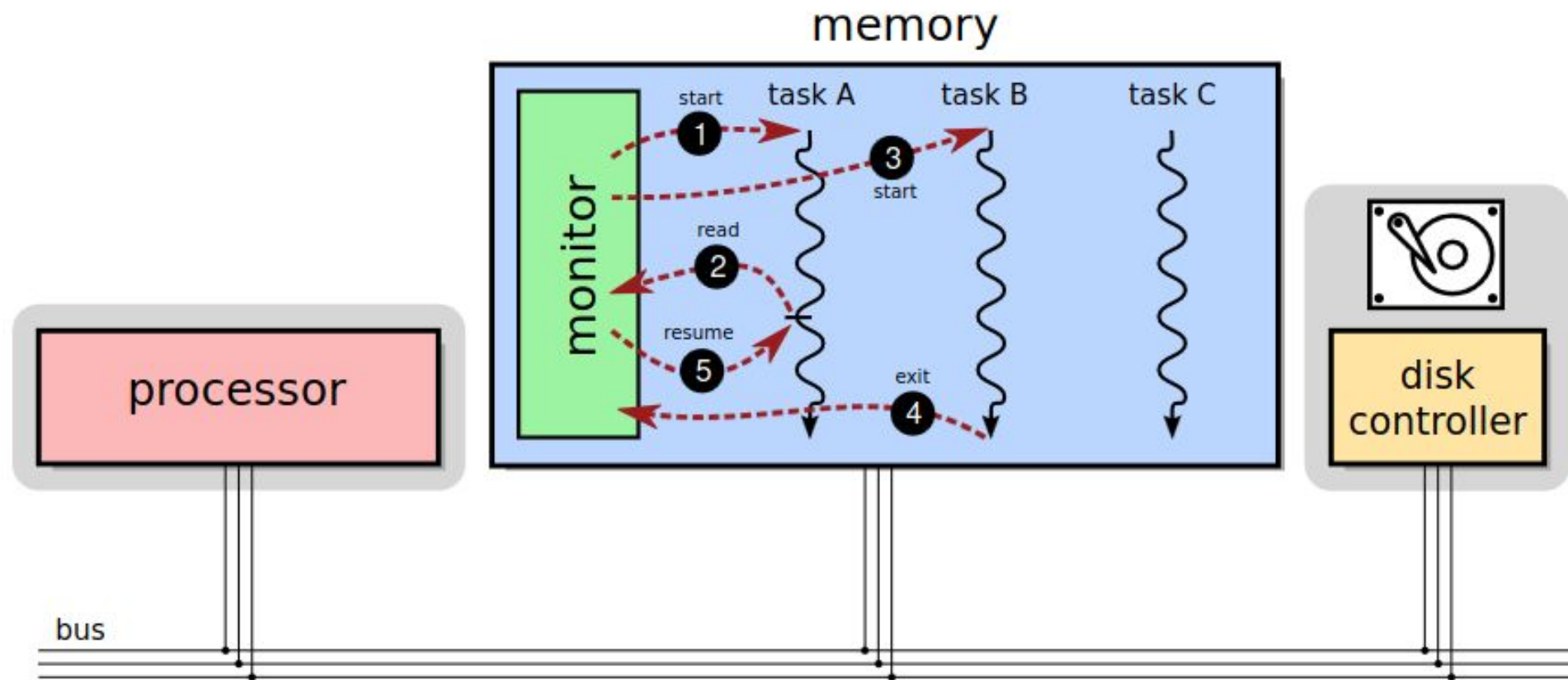


Figura 4.4: Execução de tarefas em um sistema multitarefas.

Paralelismo real



Múltiplas CPUs / núcleos

Processos executam **simultaneamente** em núcleos diferentes



Sistemas multicore

Processadores modernos possuem **vários núcleos** para execução paralela



GPUs

Executam **milhares de tarefas** simultaneamente — alto grau de paralelismo

Qual o problema da multiprogramação?

Competição

Vários processos **competem pela CPU** ao mesmo tempo

Espera por E/S

Processos precisam **aguardar eventos** externos sem desperdiçar CPU

Decisão do SO

O SO precisa decidir **quem executa e quando** — papel do escalonador

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

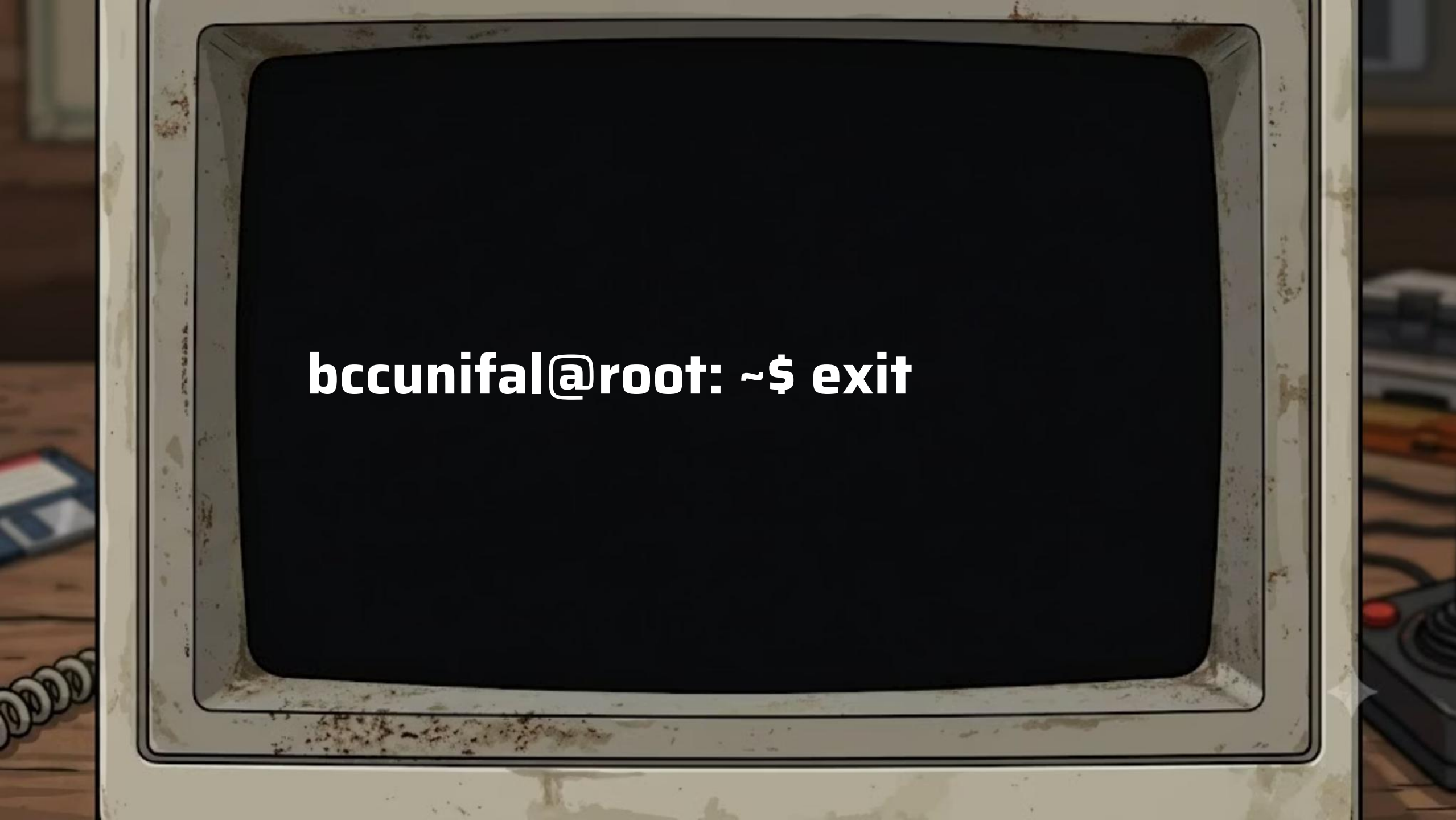
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit